Transformers & Attention

Query/Key/Value, the scaled dot-product attention formula, positional encoding, multi-head attention, and causal masking

Concept Guide 6 of 9 | Read before: Transformers & Attention Zero-to-Hero lab | Companion to the Zero-to-Hero series

---

1. Why Attention Replaces Recurrence

RNNs process a sequence one word at a time, squeezing everything seen so far through a single fixed-size hidden state — information from early words can get diluted by the time the network reaches later words. Attention lets every word look directly at every other word, all at once, with no bottleneck.

DIAGRAM — The core difference

```
  RNN:        word1 -> [mem] -> word2 -> [mem] -> ... -> word50   (info can get l
ost)
  Attention:  every word can directly access every other word, in one step
```

MENTAL MODEL

Attention answers the question 'for this word, which other words in the sentence matter, and how much?' — and it computes that answer as a set of numeric weights, fresh, for every single word.

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.1**

2. Query, Key, Value: The Search Metaphor

Attention projects each word's embedding into three different vectors, each with a distinct role.

FORMULA — The three projections

$$Q = X \mathbin{@} W_Q \text{ (this word's 'question')}$$
$$K = X \mathbin{@} W_K \text{ (every word's 'advertisement' of its content)}$$
$$V = X \mathbin{@} W_V \text{ (every word's actual content to gather)}$$

W_Q, W_K, W_V are learned weight matrices, the same shape for every word

MENTAL MODEL

Think of a library search: your Query is your search terms, each book's Key is its index-card summary, and matching a Query against Keys tells you which books' Values (their actual content) to gather and how much weight to give each one.

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.4**

3. The Attention Formula

This single formula is the heart of every modern LLM — GPT, Claude, Gemini all compute this, over and over, many times per layer.

FORMULA — Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

d_k is the dimension of the query/key vectors

WORKED EXAMPLE — Reading the formula left to right

1. Q K^T : dot product of every query with every key -> a matrix of raw similarity scores
2. / sqrt(d_k) : scale down, so scores don't grow too large and destabilize softmax
3. softmax(...) : turn each row of scores into a probability distribution (sums to 1)
4. ... V : use those weights to compute a WEIGHTED AVERAGE of the value vectors

COMMON MISUNDERSTANDINGS

- Skipping the sqrt(d_k) scaling makes dot products grow large as dimension increases, pushing softmax toward an almost one-hot ('all-or-nothing') distribution, which stalls learning — always scale.
- Attention alone has NO sense of word order — it treats the input as an unordered set until positional encoding (next section) is added.

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.5**

4. Positional Encoding

Because attention has no built-in order, a unique 'position fingerprint' is added to each word's embedding, made of sine and cosine waves at different frequencies.

FORMULA — Positional encoding

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d\_model)})$$
$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d\_model)})$$

pos = position in the sequence; i = dimension index; each position gets a unique pattern

MENTAL MODEL

Each position gets stamped with its own unique wave pattern, added directly onto the word's embedding — like a timestamp mixed into the meaning vector, so the model can tell 'cat sat mat' apart from 'mat sat cat.'

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.3**

5. Multi-Head Attention

Running one attention computation only captures one KIND of relationship between words. Multi-head attention runs several attention computations in parallel, each with its own learned Q/K/V weights, then combines their outputs.

DIAGRAM — Parallel heads, then combine

```
        head 1: Attention(Q1,K1,V1) -> output1  (maybe tracks grammar)
  in -> head 2: Attention(Q2,K2,V2) -> output2  (maybe tracks topic)     -> conca
t -> mix -> out
        head 3: Attention(Q3,K3,V3) -> output3  (maybe tracks coreference)
```

MENTAL MODEL

Like having several analysts read the same sentence, each looking for a different kind of pattern, then combining their notes into one report.

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.6**

6. Residuals, Layer Norm, and Causal Masking

Three more ingredients complete a Transformer block: residual connections (so gradients can flow through deep stacks), layer normalization (for training stability), and causal masking (so a generating model can't peek at future words).

FORMULA — A Transformer block

$$x = \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$
$$x = \text{LayerNorm}(x + \text{FeedForward}(x))$$

the '+x' is the residual connection: adding the input back to the sublayer's output

DIAGRAM — Causal masking (why GPT generates left-to-right)

```
          the  cat  sat
  the    [  ok   NO   NO ]   'the' may only see itself
  cat    [  ok   ok   NO ]   'cat' may see 'the' and itself, not 'sat'
  sat    [  ok   ok   ok ]   'sat' may see everything up to and including itself
```

COMMON MISUNDERSTANDINGS

- Without residual connections, stacking many Transformer blocks suffers the same vanishing-gradient problem as deep RNNs — residuals are structurally necessary for depth, not just an optimization.
- Causal masking is what distinguishes GPT-style (decoder-only) models from BERT-style (encoder, bidirectional) models — BERT has no mask because it isn't generating text left-to-right.

PRACTICE THIS → **Transformers_Attention_Zero_to_Hero.ipynb Ch.7-8**